TARA Processor: Complete Design Documentation

CS2300 · Group G06
Chaithanya PS (CS25B018)
Yash Tulshyan (CS25B095)

Circuit file: `G06_th6_tara.circ` · **Control ROM image:** `G06_cintrol_rom.hex` · **Generator:** `G06_th6_microcode_gen.py`

TARA is a 16-bit microprocessor featuring eight general-purpose registers (`R0–R7`), a shared 16-bit tri-state data bus, a 16-bit arithmetic logic unit (ALU) with accumulator (`ACC`) and condition flags (`Z, N, C, V`), a byte-addressed 64 KiB memory (`ram_word`, 32K × 16-bit words, little-endian), and a microprogrammed control unit. A 4-bit binary T-state counter and a 2048 × 16 control ROM drive the datapath across up to 16 T-states per instruction. In `G06_th6_tara.circ`, the control word is streamlined to 16 bits containing 13 direct control fields; four further datapath signals (`IR_LOAD`, `PC_FETCH_EN`, `BYTE_OP`, and `ALU_OPCODE`) are derived by dedicated combinational logic inside `taracpu` based on the active T-state, opcode, and instruction format. The complete ROM image is generated and validated by `G06_th6_microcode_gen.py` and embedded into `G06_th6_tara.circ`. Top-level CPU operation is provided by `taracpu` (featuring a clean tunnel architecture, eliminating wire congestion, and exposing a 32-bit execution cycle counter `CYCLES`), instantiated in `main`. This report consolidates the complete design implemented in `G06_th6_tara.circ`.

1. Interface Tables (Pin-Outs)

Every subcircuit in `G06_th6_tara.circ` and its interface ports, grouped from arithmetic primitives and storage elements to the decoder, datapath, and microcoded CPU. Arithmetic and storage blocks remain architecturally compatible, while control blocks are updated to the 16-bit control word and clean tunnel-routed CPU.

a. half_adder

Purpose: Adds two 1-bit numbers using an XOR gate and an AND gate. Basic building block of full_adder.

Label	Input / Output	Bits	Purpose
`A`	Input	1	First 1-bit operand
`B`	Input	1	Second 1-bit operand
`SUM`	Output	1	Sum bit: `A XOR B`
`COUT`	Output	1	Carry-out bit: `A AND B`

b. full_adder

Purpose: Adds two 1-bit numbers and a carry-in using two half_adder blocks and an OR gate. Four of them form adder4.

Label	Input / Output	Bits	Purpose
`A`	Input	1	First 1-bit operand
`B`	Input	1	Second 1-bit operand
`CIN`	Input	1	Carry-in bit
`SUM`	Output	1	Sum bit of `A + B + CIN`
`COUT`	Output	1	Carry-out bit of `A + B + CIN`

c. adder4

Purpose: 4-bit ripple-carry adder built from four full_adder blocks. Exposes LAST_CIN for signed overflow detection.

Label	Input / Output	Bits	Purpose
A	Input	4	First 4-bit operand
B	Input	4	Second 4-bit operand
CIN	Input	1	Carry-in to bit 0
SUM	Output	4	4-bit sum A + B + CIN
COUT	Output	1	Carry-out from bit 3
LAST_CIN	Output	1	Carry-in into bit 3 (used in MSB nibble for signed overflow calculation)

d. adder16

Purpose: 16-bit ripple-carry adder built by chaining four adder4 blocks. Forms the core addition engine of addsub16.

Label	Input / Output	Bits	Purpose
A	Input	16	First 16-bit operand
B	Input	16	Second 16-bit operand
CIN	Input	1	Carry-in to bit 0
S	Output	16	16-bit sum A + B + CIN
COUT	Output	1	Carry-out from bit 15
LAST_CIN	Output	1	Carry-in into bit 15; used to compute signed overflow

e. xor4_helper & xor16_helper

Purpose: Conditionally bitwise-inverts a 4-bit or 16-bit bus when CIN = 1 (XOR_A[i] = A[i] XOR CIN), used to form ~B for subtraction in addsub16.

Label	Input / Output	Bits	Purpose
A	Input	4 / 16	Input vector (B operand of addsub16)
CIN	Input	1	Invert control (SUB): 0 passes A unchanged, 1 outputs ~A
XOR_A	Output	4 / 16	Conditionally inverted output vector

f. addsub16

Purpose: 16-bit two's-complement adder/subtractor using xor16_helper and adder16, producing carry-out and signed overflow.

Label	Input / Output	Bits	Purpose
A	Input	16	First 16-bit operand
B	Input	16	Second 16-bit operand
SUB	Input	1	0: compute A + B; 1: compute A + (~B) + 1 = A − B
S	Output	16	16-bit addition or subtraction result
COUT	Output	1	Carry-out from bit 15 of adder16
OVF	Output	1	Signed overflow flag: COUT XOR LAST_CIN

g. shl1/2/4/8/max_helper & shr1/2/4/8/max_helper

Purpose: Fixed-distance wiring stages that shift a 16-bit word left or logically right by 1, 2, 4, or 8 bits using splitters and zero constants, plus shl_max_helper / shr_max_helper which output 0x0000 when SH[4] = 1 (shift ≥ 16).

Label	Input / Output	Bits	Purpose
A	Input	16	16-bit word entering the shift stage
Y	Output	16	16-bit word shifted by fixed stage amount (or 0x0000 for *_max_helper)

h. shl16

Purpose: 16-bit logarithmic barrel left shifter (0–31 bit positions) cascading shl1, shl2, shl4, shl8, and shl_max_helper through five 16-bit 2-to-1 multiplexers controlled by SHL_5[0..4]. Used by the ALU (for SHL) and by tara_decoder (to left-shift branch/jump offsets by 1).

Label	Input / Output	Bits	Purpose
A	Input	16	Value to be shifted left
SHL_5	Input	5	Unsigned shift count (0–31)
Y_16	Output	16	A << SHL_5 (0x0000 when SHL_5 ≥ 16)

i. shr16

Purpose: 16-bit logarithmic barrel logical right shifter (0–31 bit positions) cascading shr1, shr2, shr4, shr8, and shr_max_helper through five 16-bit 2-to-1 multiplexers controlled by SHR_5[0..4].

Label	Input / Output	Bits	Purpose
A	Input	16	Value to be shifted right logically
SHR_5	Input	5	Unsigned shift count (0–31)
Y_16	Output	16	A >> SHR_5 (logical; 0x0000 when SHR_5 ≥ 16)

j. one_bit_comp_helper

Purpose: Compares one bit of A and B while propagating greater-than, equal, and less-than results from more significant bits.

Label	Input / Output	Bits	Purpose
A, B	Input	1 each	Current bit of operands A and B
GTin, EQin, LTin	Input	1 each	Comparison state cascade from more significant bits
GT	Output	1	1 if GTin = 1, or (EQin = 1 and A > B)
EQ	Output	1	1 if EQin = 1 and A = B
LT	Output	1	1 if LTin = 1, or (EQin = 1 and A < B)

k. cmp4

Purpose: 4-bit cascading magnitude comparator built from four one_bit_comp_helper stages from MSB (bit 3) to LSB (bit 0).

Label	Input / Output	Bits	Purpose
A, B	Input	4 each	4-bit nibble operands
GTin, EQin, LTin	Input	1 each	Cascade inputs from the next more significant nibble
GT, EQ, LT	Output	1 each	Updated comparison results passed to the next lower nibble

l. cmp16

Purpose: 16-bit signed two's-complement comparator built from an MSB one_bit_comp_helper (with A[15] and B[15] inverted so negative numbers compare less than positive numbers) cascaded into four cmp4 blocks.

Label	Input / Output	Bits	Purpose
`A, B`	Input	16 each	Signed 16-bit operands
`GT`	Output	1	1 if `A > B` (signed)
`EQ`	Output	1	1 if `A = B`
`LT`	Output	1	1 if `A < B` (signed; used by ALU for SLT and flag N)

m. logic_unit

Purpose: Computes 16-bit bitwise AND, OR, XOR of A and B, and bitwise NOT of A.

Label	Input / Output	Bits	Purpose
`A, B`	Input	16 each	16-bit operands
`AND_LOGIC`	Output	16	`A & B`
`OR_LOGIC`	Output	16	`A | B`
`XOR_LOGIC`	Output	16	`A ^ B`
`NOT_LOGIC`	Output	16	`~A`

n. alu

Purpose: 16-bit Arithmetic Logic Unit executing ADD, SUB, AND, OR, XOR, NOT, SHL, SHR, and SLT selected by a 5-bit OPCODE via a 31-mux binary tree, producing result BUS_OUT and condition flags Z, N, C, V behind tri-state buffers gated by ALU_OUTPUT_ENABLE. In G06_th6_tara.circ, internal tunnel ADD connects the adder output directly to multiplexer branches.

Label	Input / Output	Bits	Purpose
`A`	Input	16	First ALU operand (fed by JMP_BZ_BN_CALL in datapath: register port QB or branch offset)
`B`	Input	16	Second ALU operand (fed by shared bus in datapath: register port QA, IMM_OUT, or PC_OUT)
`OPCODE`	Input	5	5-bit operation selector (driven by dynamically steered ALU_OPCODE)
`ALU_OUTPUT_ENABLE`	Input	1	Tri-state output enable (tied to 1 inside tara_datapath)
`BUS_OUT`	Output	16	16-bit ALU result (latched into ACC)
`Z`	Output	1	Zero flag: 1 iff `BUS_OUT == 0x0000` (from cmp16 against 0)
`N`	Output	1	Negative flag: 1 iff `BUS_OUT < 0` (signed LT from cmp16 against 0)
`C`	Output	1	Carry flag from addsub16 (ADD=9 and SUB=10 only; 0 otherwise)
`V`	Output	1	Signed overflow flag from addsub16 (ADD=9 and SUB=10 only; 0 otherwise)

o. dff

Purpose: 1-bit D flip-flop with write-enable (EN) and asynchronous reset (RST). Fundamental storage cell of register16.

Label	Input / Output	Bits	Purpose
D	Input	1	Data bit to capture
CLK	Input	1	Clock (captures on rising edge when EN = 1)
EN	Input	1	Write enable (selects D vs. feedback Q through a 2-to-1 mux)
RST	Input	1	Asynchronous clear to 0
Q	Output	1	Stored bit value

p. register16

Purpose: 16-bit parallel-load register composed of 16 dff instances. Used directly for ACC and inside PC, MAR, MDR, IR, and regfile8x16.

Label	Input / Output	Bits	Purpose
D	Input	16	16-bit data input
LOAD	Input	1	Write enable: captures D on rising CLK edge when 1
RST	Input	1	Resets all 16 bits to 0x0000
CLK	Input	1	Clock input
Q	Output	16	16-bit stored value

q. MAR

Purpose: Memory Address Register wrapping register16; holds the 16-bit byte address for ram_word and supplies ADDR0 = MAR_OP[0] to MDR for byte-lane selection.

Label	Input / Output	Bits	Purpose
MAR_IN	Input	16	Address input from the shared bus
MAR_LOAD	Input	1	Latches MAR_IN on rising clock edge
RST	Input	1	Clears MAR to 0x0000
CLK	Input	1	Clock input
MAR_OP	Output	16	Current 16-bit memory byte address

r. MDR

Purpose: Memory Data Register with integrated byte-lane multiplexers supporting full-word transfers and little-endian byte operations (LDB zero-extension onto the bus and STB byte-lane merge from the bus).

Label	Input / Output	Bits	Purpose
MDR_IN	Input	16	Input word from the shared bus
MDR_LOAD	Input	1	Latches data into internal register16 on rising clock edge
BYTE_OP	Input	1	0: full-word mode; 1: byte mode (hardware-derived in taracpu at T6 for op 7/8)
ADDR0	Input	1	MAR_OUT[0]: selects even byte lane [7:0] when 0, odd byte lane [15:8] when 1
RST	Input	1	Clears register to 0x0000
CLK	Input	1	Clock input
MDR_OUT	Output	16	Raw 16-bit stored word (drives ram_word DIN and top-level probe)
MDR_BUS	Output	16	Value driven onto bus when BUS_SOURCE = MDR: full word when BYTE_OP = 0, or zero-extended byte when BYTE_OP = 1

s. IR

Purpose: Instruction Register wrapping register16; holds the currently executing 16-bit instruction word across T-states T3..T15.

Label	Input / Output	Bits	Purpose
MDR_OUT	Input	16	Instruction word input (wired to shared bus in tara_datapath)
IR_LOAD	Input	1	Latches the bus into IR on rising clock edge (derived from TSTATE == 2)
RST	Input	1	Clears IR to 0x0000
CLK	Input	1	Clock input
IR_OUT	Output	16	Current instruction word (drives tara_decoder)

t. PC

Purpose: Program Counter holding the byte address of the next instruction, with an internal 16-bit adder (PC + 2) and 2-to-1 multiplexer selecting between sequential increment (PC + 2) and target address load from the shared bus.

Label	Input / Output	Bits	Purpose
FETCH	Input	16	Target address from the shared bus (used by JMP, BZ, BN, CALL, RET)
PC_LOAD	Input	1	Write enable: updates PC on rising clock edge when 1; holds when 0
FETCH_EN	Input	1	Next-PC mux select: 0 selects PC + 2 (fetch T1); 1 selects FETCH (bus target)
RST	Input	1	Resets PC to 0x0000
CLK	Input	1	Clock input
P_C	Output	16	Current 16-bit program counter value

u. regfile8x16

Purpose: Eight 16-bit general-purpose registers (R0–R7, where R6 is the link register and R7 is the stack pointer) with two combinational read ports (QA, QB) and one synchronous write port.

Label	Input / Output	Bits	Purpose
`RA, RB`	Input	3 each	Read register indices for port A (QA) and port B (QB)
`RD`	Input	3	Destination register index (0–7) for writes
`D`	Input	16	Write data from the shared bus
`WE`	Input	1	Write enable: writes D into R[RD] on rising clock edge
`CLK, RST`	Input	1 each	Clock and asynchronous reset (clears R0–R7 to 0x0000)
`QA, QB`	Output	16 each	Combinational read outputs R[RA] (drives shared bus) and R[RB] (feeds ALU A path)
`R0 … R7`	Output	16 each	Direct register state outputs for observation probes

v. status_reg

Purpose: 4-bit status register storing condition flags Z, N, C, and V from the ALU when STATUS_WE = 1.

Label	Input / Output	Bits	Purpose
`Z_IN, N_IN, C_IN, V_IN`	Input	1 each	Flag inputs from the ALU
`STATUS_WE`	Input	1	Write enable for all four status flip-flops
`CLK, RST`	Input	1 each	Clock and asynchronous clear
`Z, N, C, V`	Output	1 each	Latched Zero, Negative, Carry, and Overflow flags (Z and N feed ROM_ADDRESS)

w. IMM

Purpose: Immediate and offset extension unit producing a 16-bit constant for bus source 011 from IMM8 or IMM5 according to a 2-bit mode select.

Label	Input / Output	Bits	Purpose
`IMM8`	Input	8	8-bit immediate field (IR[7:0], or forced to 0xFF when control bit IMM = 1)
`IMM5`	Input	5	5-bit offset/shift field (IR[4:0])
`BIT_EXTEND`	Input	2	Mode: 00 zero-extend IMM8; 01 sign-extend IMM8; 10 IMM8 << 8; 11 sign-extend IMM5
`IMM_OUT`	Output	16	16-bit extended immediate output

x. ram_word

Purpose: Main memory wrapper around a 32K × 16 Logisim RAM primitive. Drops ADDR[0] via a splitter so consecutive even/odd byte addresses map to the same 16-bit word. Clock is inverted inside tara_datapath.

Label	Input / Output	Bits	Purpose
`ADDR`	Input	16	Byte address from MAR_OUT; ADDR[15:1] forms the 15-bit word address
`DIN`	Input	16	16-bit write data from MDR_OUT
`MEM_READ`	Input	1	Output enable for reading the addressed 16-bit word onto DOUT
`MEM_WRITE`	Input	1	Write enable for storing DIN at ADDR[15:1]
`CLK`	Input	1	RAM clock (driven by NOT(CLK) inside tara_datapath)
`DOUT`	Output	16	16-bit word read from memory

y. JMP_BZ_BN_CALL

Purpose: Combinational multiplexer block in front of ALU input A that substitutes the pre-shifted branch/jump offset (REL_8 or REL_11) for register port QB during branch target calculation states (when STATUS_WE = 0).

Label	Input / Output	Bits	Purpose
OPCODE	Input	5	Instruction opcode IR[15:11] (compared against 20, 21, 22, 23)
STATUS_WE	Input	1	When 1 (in BZ/BN state T3), inhibits offset substitution so QB = B is tested
REL_8	Input	16	sext(IR[7:0]) << 1 from tara_decoder (selected when OPCODE ∈ {20, 21})
REL_11	Input	16	sext(IR[10:0]) << 1 from tara_decoder (selected when OPCODE ∈ {22, 23})
B	Input	16	Register file read port B output (QB)
QB	Output	16	Selected 16-bit operand routed to ALU input A

z. tara_decoder

Purpose: Purely combinational instruction decoder that extracts all instruction bit-fields from IR, maps OPCODE to format 0–7 via a 32-to-1 multiplexer, and sign-extends and left-shifts REL_8 and REL_11 by 1 bit using two shl16 instances.

Label	Input / Output	Bits	Purpose
IR	Input	16	Current instruction word from IR_OUT
OPCODE	Output	5	IR[15:11]
FORMAT	Output	3	Instruction format code 0–7 (F0–F7)
RD	Output	3	IR[10:8]
RA	Output	3	IR[7:5]
RB	Output	3	IR[4:2]
IMMA8	Output	8	IR[7:0]
OFF5	Output	5	IR[4:0]
REL_8	Output	16	sext(IR[7:0]) << 1 (16-bit byte offset for BZ/BN)
REL_11	Output	16	sext(IR[10:0]) << 1 (16-bit byte offset for JMP/CALL)

aa. tara_datapath

Purpose: Integrates PC, MAR, MDR, IR, IMM, ram_word, regfile8x16, JMP_BZ_BN_CALL, alu, ACC, and status_reg around the shared 16-bit tri-state bus.

Label	Input / Output	Bits	Purpose
BUS_SOURCE	Input	3	Selects which of the 7 tri-state buffers drives the shared bus (111 = bus idle)
CLK	Input	1	System clock (inverted internally for ram_word)
MAR_LOAD, MDR_LOAD, IR_LOAD, ACC_LOAD	Input	1 each	Load enables for MAR, MDR, IR, and ACC
PC_LOAD, PC_FETCH_EN, PC_RST	Input	1 each	Program counter write enable, source select (0: +2, 1: bus target), and reset
MEM_READ, MEM_WRITE	Input	1 each	Read and write enables for ram_word
BYTE_OP	Input	1	Byte-mode control passed to MDR (hardware-derived in taracpu at T6 for LDB/STB)
REG_A, REG_B, REG_D	Input	3 each	Register file port A read index, port B read index, and destination write index
REG_WE, REG_RST	Input	1 each	Register file write enable and reset
IMM8, IMM5, IMM_BIT_EXTEND	Input	8, 5, 2	Immediate inputs and extension mode for IMM
ALU_OPCODE, STATUS_WE	Input	5, 1	ALU operation selector and status register write enable
IR_OPCODE, REL_11, REL_8	Input	5, 16, 16	Instruction opcode and pre-shifted branch offsets passed to JMP_BZ_BN_CALL
MAR_OUT, RAM_WORD_OUT, PC_OUT, MDR_OUT, IR_OUT, IMM_OUT, ALU_OUT, ACC_OUT, BUS_VALUE	Output	16 each	Observation probes for architectural registers, memory output, immediate generator, ALU, accumulator, and shared bus
R0 … R7	Output	16 each	Observation probes for general-purpose registers R0–R7
Z_OUT, N_OUT, C_OUT, V_OUT	Output	1 each	Latched status flags from status_reg

ab. tstate_counter, ROM_ADDRESS, G06_cintrol_rom, & tara_control

Purpose: The microcoded control subsystem. tstate_counter tracks the current T-state (0–15); ROM_ADDRESS packs {Z, N, OPCODE, TSTATE_OUT} into an 11-bit address; G06_cintrol_rom stores the 2048 × 16 microcode table; and tara_control unpacks the 16-bit word into 13 named control signals.

Subcircuit / Label	Input / Output	Bits	Purpose
`tstate_counter`: CLK, RST, STEP_RESET, TSTATE_HALT	Input	1 each	Clock, asynchronous reset, end-of-instruction reset to T0, and halt freeze enable
`tstate_counter`: TSTATE	Output	4	Current T-state (0–15)
`ROM_ADDRESS`: Z, N, OPCODE, TSTATE_OUT → ROM_ADDR	In → Out	1, 1, 5, 4 → 11	Forms `ROM_ADDR = (Z << 10) | (N << 9) | (OPCODE << 4) | TSTATE_OUT`
`G06_cintrol_rom`: ROM_ADDR → CONTROL	In → Out	11 → 16	2048 × 16 ROM primitive loaded from `G06_cintrol_rom.hex`
`tara_control`: Z, N, OPCODE, TSTATE_OUT	Input	1, 1, 5, 4	Status flags, instruction opcode, and current T-state passed to ROM_ADDRESS
`tara_control`: STEP_RST, MAR_LOAD, MDR_LOAD, ACC_LOAD, REG_WE, MEM_READ, MEM_WRITE, STATUS_WE, IMM, IMM_BIT_EXTEND, BUS_SOURCE, PC_LOAD, HALT	Output	16 total (13 fields)	Unpacked 16-bit control signals driving datapath and counter (Section 4)

ac. taracpu (top level) & main

Purpose: Complete top-level TARA processor circuit in G06_th6_tara.circ. Combines tstate_counter, tara_control, tara_decoder, tara_datapath, format-based register-index steering, and dedicated combinational generation of IR_LOAD, PC_FETCH_EN, BYTE_OP, and ALU_OPCODE. Includes an internal 32-bit execution cycle counter. The top-level circuit main instantiates taracpu, system clock, and top-level probes.

Label (taracpu)	Input / Output	Bits	Purpose
`CLK`	Input	1	Master processor clock
`RST`	Input	1	Master processor reset (clears counter, registers, and cycle counter; preserves RAM)
`TSTATE`	Output	4	Current T-state (0x0–0xF)
`PC`	Output	16	Program counter value (byte address of next instruction)
`R0 … R7`	Output	16 each	General-purpose register values R0 through R7
`CYCLES`	Output	32	Total executed clock cycles since reset (displayed as unsigned decimal in main)

2. Instruction Set & Formats

TARA implements 27 architectural instructions spanning arithmetic, logical, memory, branching, subroutine call/return, and stack operations. Total cycles equals the number of executed T-states per instruction, including the 3 common fetch states (T0–T2). The instruction opcode is always `IR[15:11]`.

Instr	Opcode	Format	Operation / Semantics	Cycles
`NOP`	0 (0x00)	F0	No operation	4 (T0–T3)
`HLT`	1 (0x01)	F0	Freeze T-state counter at T3	4 to halt
`MOV`	2 (0x02)	F2	`Rd ← Rs`	4 (T0–T3)
`LIL`	3 (0x03)	F3	`Rd ← zext(imm8)`	4 (T0–T3)
`LIH`	4 (0x04)	F3	`Rd ← (imm8 << 8) | (Rd & 0x00FF)`	7 (T0–T6)
`LDW`	5 (0x05)	F4	`Rd ← Mem16[Rb + sext(off5)]` (word load)	7 (T0–T6)
`STW`	6 (0x06)	F4	`Mem16[Rb + sext(off5)] ← Rs` (word store)	7 (T0–T6)
`LDB`	7 (0x07)	F4	`Rd ← zext(Mem8[Rb + sext(off5)])` (byte load, lane = MAR[0])	7 (T0–T6)
`STB`	8 (0x08)	F4	`Mem8[Rb + sext(off5)] ← Rs[7:0]` (read-modify-write byte store)	8 (T0–T7)
`ADD`	9 (0x09)	F1	`Rd ← Ra + Rb`	5 (T0–T4)
`SUB`	10 (0x0A)	F1	`Rd ← Ra − Rb`	5 (T0–T4)
`ADDI`	11 (0x0B)	F3	`Rd ← Rd + sext(imm8)`	5 (T0–T4)
`MUL`	12 (0x0C)	—	Unimplemented (executes SAFE idle word at T3, behaves as 4-cycle NOP)	4 (T0–T3)
`AND`	13 (0x0D)	F1	`Rd ← Ra & Rb`	5 (T0–T4)
`OR`	14 (0x0E)	F1	`Rd ← Ra | Rb`	5 (T0–T4)
`XOR`	15 (0x0F)	F1	`Rd ← Ra ^ Rb`	5 (T0–T4)
`NOT`	16 (0x10)	F2	`Rd ← ~Rs`	5 (T0–T4)
`SHL`	17 (0x11)	F3	`Rd ← Rd << imm8[4:0]` (logical left shift)	5 (T0–T4)
`SHR`	18 (0x12)	F3	`Rd ← Rd >> imm8[4:0]` (logical right shift)	5 (T0–T4)
`SLT`	19 (0x13)	F1	`Rd ← (Ra < Rb) ? 1 : 0` (signed 16-bit comparison)	5 (T0–T4)
`BZ`	20 (0x14)	F5	`if (Rt == 0) PC ← PC + sext(rel8) × 2`	6 taken / 5 not
`BN`	21 (0x15)	F5	`if (Rt < 0) PC ← PC + sext(rel8) × 2`	6 taken / 5 not
`JMP`	22 (0x16)	F6	`PC ← PC + sext(rel11) × 2`	5 (T0–T4)
`CALL`	23 (0x17)	F6	`R6 ← PC; PC ← PC + sext(rel11) × 2`	6 (T0–T5)
`RET`	24 (0x18)	F0	`PC ← R6`	4 (T0–T3)
`PUSH`	25 (0x19)	F7	`MDR ← Rs; R7 ← R7 − 2; Mem16[R7] ← MDR`	9 (T0–T8)
`POP`	26 (0x1A)	F7	`MDR ← Mem16[R7]; R7 ← R7 + 2; Rd ← MDR`	10 (T0–T9)

Note: Opcodes 27–31 are unassigned; their ROM entries hold `SAFE = 0x3801` at T3, executing as 4-cycle NOPs without modifying state.

Note on POP R7: POP always increments the stack pointer (T6, T8) and writes the popped word to `Rd` last (T9). When `Rd = R7`, the final write at T9 overwrites the incremented SP, so the pointer is *not* advanced by 2; instead `R7 ← Mem16[old R7]`, i.e. SP is loaded with the word that was on top of the stack. The net effect is that of a plain load of SP from the stack, and the +2 update is lost.

Instruction Formats & Register-Port Steering

In `tara_datapath`, register-file Port B (`QB`, selected by `REG_B`) feeds ALU input A (via `JMP_BZ_BN_CALL`), while register-file Port A (`QA`, selected by `REG_A`) drives the shared bus (when `BUS_SOURCE = 100`), which feeds ALU input B. `REG_D` selects the destination register written when `REG_WE = 1`. Inside `taracpu`, combinational steering decodes `FORMAT` (`FMT[0..7]`) to steer `RA, RB, RD`, `R6` (0x6, link register), and `R7` (0x7, stack pointer) onto the three register ports:

- **REG_A:** `0x6` if `FMT[0]` (RET), else `RD` if `FMT[3] | FMT[4] | FMT[7]` (stores/push/imm), else `RB`.
- **REG_B:** `0x7` if `FMT[7]` (PUSH/POP), else `RD` if `FMT[3] | FMT[5]` (LIH/ADDI/shifts/branches), else `RA`.
- **REG_D:** `0x7` if `FMT[7] & (BUS_SOURCE == 5)` (SP write-back from ACC), else `0x6` if `FMT[6]` (CALL link reg), else `RD`.

Format	Bit Layout (15..0)	Instructions	REG_A (Port A → Bus)	REG_B (Port B → ALU A)	REG_D (Write Port)
F0 (000)	`op | 00000000000`	NOP, HLT, RET	`R6` (forced; for RET)	RA (unused)	RD (unused)
F1 (001)	`op | rd | rsA | rsB | 00`	ADD, SUB, AND, OR, XOR, SLT	`IR[4:2]` (RB = rsB)	`IR[7:5]` (RA = rsA)	`IR[10:8]` (RD = rd)
F2 (010)	`op | rd | rs | 00000`	MOV, NOT	`IR[4:2]` (unused)	`IR[7:5]` (RA = rs)	`IR[10:8]` (RD = rd)
F3 (011)	`op | rd | imm8`	LIL, LIH, ADDI, SHL, SHR	`IR[10:8]` (RD, unused)	`IR[10:8]` (RD = rd)	`IR[10:8]` (RD = rd)
F4 (100)	`op | rdata | rbase | off5`	LDW, STW, LDB, STB	`IR[10:8]` (RD = rdata for stores)	`IR[7:5]` (RA = rbase)	`IR[10:8]` (RD = rdata for loads)
F5 (101)	`op | rtest | rel8`	BZ, BN	`IR[4:2]` (unused)	`IR[10:8]` (RD = rtest)	`IR[10:8]` (unused)
F6 (110)	`op | rel11`	JMP, CALL	`IR[4:2]`	`IR[7:5]` (unused)	`R6` (forced; link reg for CALL)
F7 (111)	`op | rstk | 00000000`	PUSH, POP	`IR[10:8]` (RD = rstk for PUSH), R7 (forced; for POP)	`R7` (forced; SP)	`R7` when BSRC=ACC (SP update), else `RD` (POP)

3. Select Implementation Details

- **a. adder16 & addsub16:** `adder16` chains four `adder4` blocks (16 `full_adder` cells). `LAST_CIN` (carry into bit 15) is exposed so `addsub16` computes signed overflow as `OVF = COUT XOR LAST_CIN`. Subtraction uses `xor16_helper` to bitwise invert B and feeds `SUB = 1` into `CIN` to compute `A + (~B) + 1 = A − B`.
- **b. shl16 & shr16:** Both shifters use five cascaded 16-bit 2-to-1 multiplexers controlled by `SH[0..4]`. Stages 0–3 route through splitter-based fixed shifts of 1, 2, 4, and 8 bits; stage 4 routes through `shl_max_helper / shr_max_helper`, which forces the output to `0x0000` whenever the shift count is 16–31 (`SH[4] = 1`).
- **c. cmp16:** Built from four 4-bit `cmp4` blocks (16 `one_bit_comp_helper` cells) plus an MSB comparator cell on bit 15 whose inputs pass through NOT gates (~A[15] vs. ~B[15]). Inverting the sign bits transforms unsigned magnitude comparison into signed two's-complement comparison.
- **d. alu:** Combines two `addsub16` instances (one with `SUB=0` for ADD, one with `SUB=1` for SUB), `logic_unit`, `shl16`, `shr16`, and four `cmp16` blocks. A 5-level binary tree of 31 multiplexers selects the operation from `OPCODE`. Condition flags Z and N are computed by comparing the selected result against zero (`Z = EQ`, `N = LT`). Carry (C) and overflow (V) are gated to 0 for all instructions except ADD and SUB. In `G06_th6_tara.circ`, tunnel `ADD` connects the adder output directly to relevant multiplexer inputs.

- **e. regfile8x16:** Contains eight `register16` blocks. A 3-to-8 decoder on RD (enabled by WE) drives the individual register LOAD pins. Two 3-to-8 decoders on RA and RB control 16-bit tri-state buffers driving read ports QA and QB combinationally.
- **f. ram_word & MDR:** `ram_word` wraps a 32K × 16 Logisim RAM whose address input receives `ADDR[15:1]` (dropping `ADDR[0]`) and whose clock is inverted inside `tara_datapath`. Byte-lane extraction for LDB and byte-lane merging for STB are performed inside `MDR` using five 8-bit multiplexers controlled by `BYTE_OP` and `ADDR0 = MAR_OUT[0]` (detailed in Section 10).
- **g. IMM:** Uses three Bit Extenders and a splitter feeding a 4-to-1 16-bit multiplexer selected by `BIT_EXTEND`: 00 = zero-extend IMM8, 01 = sign-extend IMM8, 10 = IMM8 << 8, 11 = sign-extend IMM5.
- **h. tara_decoder & JMP_BZ_BN_CALL:** `tara_decoder` splits IR into bitfields, decodes FORMAT via a 32-to-1 mux, and sign-extends and left-shifts `REL_8` and `REL_11` by 1 bit using two `shl16` instances. Inside `tara_datapath`, `JMP_BZ_BN_CALL` substitutes `REL_8` (for BZ/BN) or `REL_11` (for JMP/CALL) onto ALU input A whenever `STATUS_WE = 0`.
- **i. tstate_counter, tara_control, & taracpu:** tstate_counter is a 4-bit Logisim counter. STEP_RESET, gated by ~TSTATE_HALT, synchronously loads 0. Counting is disabled while TSTATE_HALT or STEP_RESET is asserted. RST clears it asynchronously. `tara_control` addresses the 2048 × 16 `G06_cintrol_rom` with `{Z, N, OPCODE, TSTATE}`. `taracpu` connects the control unit to the datapath using clean named tunnels and drives top-level probes and the 32-bit cycle counter.
- **j. Dedicated hardware control signal generation in taracpu:** Rather than expanding ROM width, four signals are derived combinationally inside `taracpu`:
  - `IR_LOAD = (TSTATE == 2)`: captures instruction at fetch state T2.
  - `PC_FETCH_EN = (TSTATE != 1)`: 0 at T1 (sequential PC ← PC + 2), 1 in execution states (PC ← bus target for jumps/branches/calls/ret).
  - `BYTE_OP = (OPCODE ∈ {7, 8}) & (TSTATE == 6)`: activates byte-lane handling in MDR during T6 of LDB and STB.
  - `ALU_OPCODE`: driven by instruction opcode, with dynamic overrides for:
    - Op 4 (LIH): T3 → 13 (AND), T5 → 14 (OR)
    - Op 20/21 (BZ/BN): T3 → 13 (AND), so the flag test computes Rt & 0xFFFF
    - Op 26 (POP): T3 → 13 (AND), T6/T8 → 10 (SUB)

4. Control Word Structure & Signals

The control ROM contains 2048 words of 16 bits, indexed by an 11-bit address: `ROM_ADDR = (Z << 10) | (N << 9) | (OPCODE << 4) | TSTATE`. Instructions that do not depend on condition flags store identical words across all four (Z, N) quadrants. Unused ROM entries hold the idle word `SAFE = 0x3801` (`BUS_SOURCE = 111`, `STEP_RST = 1`).

Bit(s)	Field	Width	Function & Encoding	Idle Value
0	`STEP_RST`	1	1: resets `tstate_counter` to T0 on next rising clock edge (end of instruction)	1
1	`MAR_LOAD`	1	1: loads MAR from the shared bus	0
2	`MDR_LOAD`	1	1: loads MDR from the shared bus (full word or merged byte lane when `BYTE_OP=1`)	0
3	`ACC_LOAD`	1	1: latches ALU output `BUS_OUT` into ACC	0
4	`REG_WE`	1	1: writes the shared bus value into `R[REG_D]`	0
5	`MEM_READ`	1	1: enables `ram_word` read output onto bus source 110 (RAM)	0
6	`MEM_WRITE`	1	1: writes `MDR_OUT` into `ram_word` at word address `MAR_OUT[15:1]`	0

Bit(s)	Field	Width	Function & Encoding	Idle Value
7	STATUS_WE	1	1: latches ALU flags Z, N, C, V into status_reg (and disables JMP_BZ_BN_CALL offset override)	0
8	IMM	1	0: passes decoder IMMA8 to IMM8; 1: forces IMM8 = 0xFF via top-level mux	0
10..9	IMM_BIT_EXTEND	2	Extension mode: 00 zero-ext imm8, 01 sign-ext imm8, 10 imm8<<8, 11 sign-ext imm5	00
13..11	BUS_SOURCE	3	Selects which unit drives the shared 16-bit bus (0–6 active, 7 = idle/off)	111 (off)
14	PC_LOAD	1	1: updates PC on rising clock edge	0
15	HALT	1	1: freezes tstate_counter (asserted by HLT at T3)	0

Immediate Extension Modes (IMM_BIT_EXTEND)

Code	Symbol	Output IMM_OUT	Used By
00 (0)	EXT_ZERO8	zext(IMM8) (0x00FF if IMM=1)	LIL, LIH (T3 mask)
01 (1)	EXT_SIGN8	sext(IMM8) (0xFFFF if IMM=1)	ADDI, BZ/BN (T3 test), PUSH/POP (±1 step)
10 (2)	EXT_SHL8	IMM8 << 8 ({IMM8, 0x00})	LIH (T5 high-byte merge)
11 (3)	EXT_SIGN5	sext(IMM5) (sext(IR[4:0]))	LDW, STW, LDB, STB, SHL, SHR

Bus Source Multiplexer Encoding (BUS_SOURCE)

Code	Symbol	Bus Driver	Notes
000 (0)	BUS_PC	Program Counter (PC_OUT)	Used during fetch T0, branch/jump target calculation, CALL
001 (1)	BUS_MDR	Memory Data Register (MDR_BUS)	Used during fetch T2 (IR load), LDW/LDB, POP, STW/STB
010 (2)	BUS_IR	Instruction Register (IR_OUT)	Available for direct instruction word transfers
011 (3)	BUS_IMM	Immediate Generator (IMM_OUT)	Used by LIL, LIH, ADDI, shifts, base+offset addressing, constants
100 (4)	BUS_REG	Register File Port A (QA = R[REG_A])	Feeds ALU input B for R-type ops; source for stores/PUSH; RET
101 (5)	BUS_ACC	Accumulator (ACC_OUT)	Writes ALU result to destination register or MAR; PC target load
110 (6)	BUS_RAM	Memory Data Output (RAM_WORD_OUT)	Direct memory read into MDR during fetch T1 and memory loads
111 (7)	BUS_OFF	None (all bus drivers disabled)	Shared bus idle; asserted in SAFE idle word and HLT

Hardware-Derived Datapath Signals (Generated in taracpu)

To maintain a clean 16-bit ROM word, four control signals are derived by combinational logic inside taracpu rather than stored in the microcode ROM:

Signal	Generating Condition	Function & Architectural Rationale
IR_LOAD	TSTATE == 2	Captures the instruction word from MDR into IR at the end of the fetch sequence. Active exclusively during T2 across all instructions.

Signal	Generating Condition	Function & Architectural Rationale
PC_FETCH_EN	TSTATE != 1	Controls the next-PC multiplexer: 0 during T1 (selects sequential PC ← PC + 2), and 1 during execution states (selects branch/jump/subroutine target from the shared bus).
BYTE_OP	(OPCODE ∈ {7, 8}) & (TSTATE == 6)	Enables byte-lane selection in MDR: zero-extends the addressed byte onto MDR_BUS for LDB (op 7) at T6, and merges Rs[7:0] into the addressed byte lane for STB (op 8) at T6.
ALU_OPCODE	OPCODE with dynamic overrides: • Op 4 (LIH): T3 → 13 (AND), T5 → 14 (OR) • Op 20/21 (BZ/BN): T3 → 13 (AND), so the flag test computes Rt & 0xFFFF • Op 26 (POP): T3 → 13 (AND), T6/T8 → 10 (SUB)	Selects the active ALU operation. By defaulting to IR[15:11], all arithmetic/logical instructions directly command the ALU. Special multi-step operations (LIH masking/merging, BZ/BN flag testing, and POP stack pointer incrementing) are dynamically steered.

5. Fetch Microsequence (T0–T2)

Every instruction execution begins with the identical three-state fetch sequence (T0–T2), replicated across all 32 opcodes and all 4 condition code combinations in the ROM because IR still holds the preceding instruction during fetch:

State	Hex Word	Control Signals Asserted	Micro-operation & Timing Description
T0	0x0002	BUS_SOURCE=000 (PC), MAR_LOAD=1	MAR ← PC Drives the current program counter onto the bus and latches it into MAR on the rising clock edge.
T1	0x7024	BUS_SOURCE=110 (RAM), MEM_READ=1, MDR_LOAD=1, PC_LOAD=1, (PC_FETCH_EN=0)	MDR ← Mem16[MAR]; PC ← PC + 2 Enables memory read at address MAR. The inverted RAM clock (~CLK) latches RAM output at the mid-cycle falling edge. On the rising edge, MDR captures the instruction and PC increments by 2.
T2	0x0800	BUS_SOURCE=001 (MDR), (IR_LOAD=1 derived)	IR ← MDR Drives MDR onto the bus. Hardware detector in taracpu asserts IR_LOAD at T2, latching the instruction into IR at the rising edge to begin execution at T3.

Timing Hazard Resolution: Clocking ram_word on the inverted clock ensures memory data is fully stable on the shared bus well before the rising clock edge of T1, allowing MDR and PC to update simultaneously without race conditions or extra wait states.

6. Complete Microcode Table

Complete 16-bit control words generated by `G06_th6_microcode_gen.py` and stored in `G06_cintrol_rom.hex`. Addresses are shown for the `Z = 0, N = 0` quadrant (`0x000–0x1FF`) along with branch-taken addresses in quadrants `Z = 1` (`0x400–0x5FF`) and `N = 1` (`0x200–0x3FF`).

Instr (Op)	State	ROM Addr	Control Word	Control Signals Asserted	Micro-operation
FETCH (all ops)	T0	`0x..0`	`0x0002`	`BUS=PC, MAR_LOAD`	`MAR ← PC`
	T1	`0x..1`	`0x7024`	`BUS=RAM, MEM_R, MDR_LOAD, PC_LOAD`	`MDR ← Mem16[MAR]; PC ← PC + 2`
	T2	`0x..2`	`0x0800`	`BUS=MDR, (IR_LOAD derived)`	`IR ← MDR`
NOP (0)	T3	`0x003`	`0x3801`	`STEP_RST, BUS=OFF`	Idle; return to T0
HLT (1)	T3	`0x013`	`0xb800`	`HALT, BUS=OFF`	Freeze T-state counter at T3
MOV (2)	T3	`0x023`	`0x2011`	`REG_WE, BUS=REG`	`Rd ← Rs`
LIL (3)	T3	`0x033`	`0x1811`	`STEP_RST, REG_WE, BUS=IMM, EXT=ZERO8`	`Rd ← zext(imm8)`
LIH (4)	T3	`0x043`	`0x1908`	`ACC_LOAD, IMM, BUS=IMM, (ALU_OP=13)`	`ACC ← Rd & 0x00FF`
	T4	`0x044`	`0x2810`	`REG_WE, BUS=ACC`	`Rd ← ACC` (clear high byte)
	T5	`0x045`	`0x1c08`	`ACC_LOAD, EXT=SHL8, BUS=IMM, (ALU_OP=14)`	`ACC ← Rd | (imm8 << 8)`
	T6	`0x046`	`0x2811`	`STEP_RST, REG_WE, BUS=ACC`	`Rd ← ACC`
LDW (5)	T3	`0x053`	`0x1e08`	`ACC_LOAD, EXT=SIGN5, BUS=IMM`	`ACC ← Rb + sext(off5)`
	T4	`0x054`	`0x2802`	`MAR_LOAD, BUS=ACC`	`MAR ← ACC`
	T5	`0x055`	`0x3024`	`MDR_LOAD, MEM_READ, BUS=RAM`	`MDR ← Mem16[MAR]`
	T6	`0x056`	`0x0811`	`STEP_RST, REG_WE, BUS=MDR`	`Rd ← MDR`
STW (6)	T3	`0x063`	`0x1e08`	`ACC_LOAD, EXT=SIGN5, BUS=IMM`	`ACC ← Rb + sext(off5)`
	T4	`0x064`	`0x2802`	`MAR_LOAD, BUS=ACC`	`MAR ← ACC`
	T5	`0x065`	`0x2004`	`MDR_LOAD, BUS=REG`	`MDR ← Rs`
	T6	`0x066`	`0x0841`	`STEP_RST, MEM_WRITE, BUS=MDR`	`Mem16[MAR] ← MDR`
LDB (7)	T3	`0x073`	`0x1e08`	`ACC_LOAD, EXT=SIGN5, BUS=IMM`	`ACC ← Rb + sext(off5)`
	T4	`0x074`	`0x2802`	`MAR_LOAD, BUS=ACC`	`MAR ← ACC`
	T5	`0x075`	`0x3024`	`MDR_LOAD, MEM_READ, BUS=RAM`	`MDR ← Mem16[MAR]`
	T6	`0x076`	`0x0811`	`STEP_RST, REG_WE, BUS=MDR, (BYTE_OP derived)`	`Rd ← zext(MDR byte MAR[0])`

Instr (Op)	State	ROM Addr	Control Word	Control Signals Asserted	Micro-operation
STB (8)	T3	0x083	0x1e08	ACC_LOAD, EXT=SIGN5, BUS=IMM	ACC ← Rb + sext(off5)
	T4	0x084	0x2802	MAR_LOAD, BUS=ACC	MAR ← ACC
	T5	0x085	0x3024	MDR_LOAD, MEM_READ, BUS=RAM	MDR ← Mem16[MAR]
	T6	0x086	0x2004	MDR_LOAD, BUS=REG, (BYTE_OP derived)	MDR[byte MAR[0]] ← Rs[7:0]
	T7	0x087	0x0841	STEP_RST, MEM_WRITE, BUS=MDR	Mem16[MAR] ← MDR
ADD (9)	T3	0x093	0x2008	ACC_LOAD, BUS=REG	ACC ← Ra + Rb
	T4	0x094	0x2811	STEP_RST, REG_WE, BUS=ACC	Rd ← ACC
SUB (10)	T3	0x0a3	0x2008	ACC_LOAD, BUS=REG	ACC ← Ra − Rb
	T4	0x0a4	0x2811	STEP_RST, REG_WE, BUS=ACC	Rd ← ACC
ADDI (11)	T3	0x0b3	0x1a08	ACC_LOAD, EXT=SIGN8, BUS=IMM	ACC ← Rd + sext(imm8)
	T4	0x0b4	0x2811	STEP_RST, REG_WE, BUS=ACC	Rd ← ACC
MUL (12)	T3	0x0c3	0x3801	STEP_RST, BUS=OFF	Unimplemented; executes SAFE idle word
AND (13)	T3	0x0d3	0x2008	ACC_LOAD, BUS=REG	ACC ← Ra & Rb
	T4	0x0d4	0x2811	STEP_RST, REG_WE, BUS=ACC	Rd ← ACC
OR (14)	T3	0x0e3	0x2008	ACC_LOAD, BUS=REG	ACC ← Ra | Rb
	T4	0x0e4	0x2811	STEP_RST, REG_WE, BUS=ACC	Rd ← ACC
XOR (15)	T3	0x0f3	0x2008	ACC_LOAD, BUS=REG	ACC ← Ra ^ Rb
	T4	0x0f4	0x2811	STEP_RST, REG_WE, BUS=ACC	Rd ← ACC
NOT (16)	T3	0x103	0x2008	ACC_LOAD, BUS=REG	ACC ← ~Rs
	T4	0x104	0x2811	STEP_RST, REG_WE, BUS=ACC	Rd ← ACC
SHL (17)	T3	0x113	0x1e08	ACC_LOAD, EXT=SIGN5, BUS=IMM	ACC ← Rd << imm8[4:0]
	T4	0x114	0x2811	STEP_RST, REG_WE, BUS=ACC	Rd ← ACC
SHR (18)	T3	0x123	0x1e08	ACC_LOAD, EXT=SIGN5, BUS=IMM	ACC ← Rd >> imm8[4:0]
	T4	0x124	0x2811	STEP_RST, REG_WE, BUS=ACC	Rd ← ACC
SLT (19)	T3	0x133	0x2008	ACC_LOAD, BUS=REG	ACC ← (Ra < Rb) ? 1 : 0
	T4	0x134	0x2811	STEP_RST, REG_WE, BUS=ACC	Rd ← ACC

Instr (Op)	State	ROM Addr	Control Word	Control Signals Asserted	Micro-operation
BZ (20)	T3	0x143	0x1b80	STATUS_WE, IMM, EXT=SIGN8, BUS=IMM	STATUS ← flags(Rt & 0xFFFF)
	T4 (Z=0)	0x144	0x3801	STEP_RST, BUS=OFF	Not taken: return to T0
	T4 (Z=1)	0x544	0x0008	ACC_LOAD, BUS=PC	Taken: ACC ← PC + sext(rel8)×2
	T5 (Z=1)	0x545	0x6801	STEP_RST, PC_LOAD, BUS=ACC	Taken: PC ← ACC
BN (21)	T3	0x153	0x1b80	STATUS_WE, IMM, EXT=SIGN8, BUS=IMM	STATUS ← flags(Rt & 0xFFFF)
	T4 (N=0)	0x154	0x3801	STEP_RST, BUS=OFF	Not taken: return to T0
	T4 (N=1)	0x354	0x0008	ACC_LOAD, BUS=PC	Taken: ACC ← PC + sext(rel8)×2
	T5 (N=1)	0x355	0x6801	STEP_RST, PC_LOAD, BUS=ACC	Taken: PC ← ACC
JMP (22)	T3	0x163	0x0008	ACC_LOAD, BUS=PC	ACC ← PC + sext(rel11)×2
	T4	0x164	0x6801	STEP_RST, PC_LOAD, BUS=ACC	PC ← ACC
CALL (23)	T3	0x173	0x0008	ACC_LOAD, BUS=PC	ACC ← PC + sext(rel11)×2
	T4	0x174	0x0010	REG_WE, BUS=PC	R6 ← PC (link register)
	T5	0x175	0x6801	STEP_RST, PC_LOAD, BUS=ACC	PC ← ACC
RET (24)	T3	0x183	0x6001	STEP_RST, PC_LOAD, BUS=REG	PC ← R6
PUSH (25)	T3	0x193	0x2004	MDR_LOAD, BUS=REG	MDR ← Rs
	T4	0x194	0x1b08	ACC_LOAD, IMM, EXT=SIGN8, BUS=IMM	ACC ← R7 + 0xFFFF (SP − 1)
	T5	0x195	0x2810	REG_WE, BUS=ACC	R7 ← ACC
	T6	0x196	0x1b08	ACC_LOAD, IMM, EXT=SIGN8, BUS=IMM	ACC ← R7 + 0xFFFF (SP − 2)
	T7	0x197	0x2812	MAR_LOAD, REG_WE, BUS=ACC	R7 ← ACC; MAR ← ACC
	T8	0x198	0x0841	STEP_RST, MEM_WRITE, BUS=MDR	Mem16[MAR] ← MDR
POP (26)	T3	0x1a3	0x2002	MAR_LOAD, BUS=REG	MAR ← R7
	T4	0x1a4	0x3024	MDR_LOAD, MEM_READ, BUS=RAM	MDR ← Mem16[MAR]
	T5	0x1a5	0x1b08	ACC_LOAD, IMM, EXT=SIGN8, BUS=IMM, (ALU_OP=10)	ACC ← R7 − 0xFFFF (SP + 1)
	T6	0x1a6	0x2810	REG_WE, BUS=ACC	R7 ← ACC
	T7	0x1a7	0x1b08	ACC_LOAD, IMM, EXT=SIGN8, BUS=IMM, (ALU_OP=10)	ACC ← R7 − 0xFFFF (SP + 2)
	T8	0x1a8	0x2810	REG_WE, BUS=ACC	R7 ← ACC
	T9	0x1a9	0x0811	STEP_RST, REG_WE, BUS=MDR	Rd ← MDR

POP R7: if `Rd = R7`, the T9 write (`Rd ← MDR`) targets the same register that T6 and T8 just incremented, so the incremented SP is discarded and `R7` ends up holding `Mem16[old R7]` (the popped value) rather than `old R7 + 2`.

7. Key Design Decisions

1. **Hierarchical ripple-carry adder with LAST_CIN overflow tap:** `adder16` is constructed from four `adder4` blocks, each containing four `full_adder` cells (each built from two `half_adder` cells). Bringing out `LAST_CIN` (carry into bit 15) allows `addsub16` to compute two's-complement signed overflow with a single XOR gate: `OVF = COUT XOR LAST_CIN`.
2. **Logarithmic barrel shifters with SH[4] max-shift guard:** Rather than chaining 15 1-bit shifters, `shl16` and `shr16` cascade fixed splitter stages of 1, 2, 4, and 8 bits selected by `SH[0..3]`, followed by `shl_max_helper / shr_max_helper` controlled by `SH[4]`, which forces the output to `0x0000` whenever the shift count is 16–31.
3. **Signed 16-bit comparator via MSB sign inversion:** In two's-complement arithmetic, a negative number has MSB 1 and a non-negative number has MSB 0. `cmp16` inverts `A[15]` and `B[15]` before feeding the MSB `one_bit_comp_helper`, and cascades its GT, EQ, LT outputs through four 4-bit `cmp4` blocks from bits 15..12 down to 3..0.
4. **Binary multiplexer tree and ALU_OUTPUT_ENABLE in the ALU:** The ALU decodes its 5-bit `ALU_OPCODE` using a 5-level binary tree of 31 2-to-1 multiplexers. Carry (C) and overflow (V) are gated with 5-bit equality comparators so they can only be 1 during ADD (opcode 9) and SUB (opcode 10). Internal tunnel `ADD` connects the adder directly to relevant tree inputs.
5. **Compositional register file with combinational tri-state read ports:** `regfile8x16` is built from eight verified `register16` blocks. Both read ports (QA, QB) are purely combinational via 3-to-8 decoders and 16-bit controlled buffers, while writes occur on the rising clock edge. Temporary loopback multiplexers were removed once the shared bus was introduced.
6. **Little-endian byte addressing and word alignment in ram_word:** A splitter inside `ram_word` drops `ADDR[0]` and passes `ADDR[15:1]` to the 15-bit address port of the 32K × 16 RAM primitive. Even byte addresses (`ADDR[0] = 0`) correspond to low byte [7:0] and odd byte addresses (`ADDR[0] = 1`) to high byte [15:8], matching the byte order of LIL/LIH.
7. **Inverted RAM clock for 3-cycle instruction fetch:** Because the Logisim RAM primitive latches synchronously, clocking it on the rising edge caused MDR to capture stale data during T1. Inverting the RAM clock with a NOT gate inside `tara_datapath` makes `ram_word` latch on the falling edge mid-way through T1 (and T5 of loads/POP), so valid memory data is present on the bus before the rising edge at the end of the state.
8. **Encoded 3-bit BUS_SOURCE and dedicated accumulator ACC:** A 3-to-8 decoder on `BUS_SOURCE` guarantees that at most one of the seven bus sources drives the shared bus in any T-state (111 = bus idle). The ALU output is captured in ACC rather than driving the bus directly, preventing bus contention between ALU input B (on the bus) and the ALU output.
9. **Swapped register-file read port routing (RA → REG_B → ALU A, RB → REG_A → Bus → ALU B):** In `tara_datapath`, ALU input A comes from register port B (QB) and ALU input B comes from the shared bus (driven by register port A, QA). Routing the instruction's first source operand RA to `REG_B` and second source operand RB to `REG_A` ensures non-commutative instructions (SUB, SLT, SHL, SHR, NOT) receive their first operand on ALU input A.
10. **Sign-extended 5-bit offset mode (EXT_SIGN5 = 11) in IMM:** Mode 11 of `IMM` sign-extends the 5-bit field `IR[4:0]` to 16 bits, allowing LDW, STW, LDB, and STB to use signed byte offsets (−16 to +15). Because SHL and SHR only examine B[4:0] sharing mode 11 across all three requires no extra hardware or ROM bits.
11. **Triple-duty IMM = 1 override mux (0xFF):** Setting control bit `IMM = 1` replaces `IMMA8` with `0xFF` before `IMM`. Combined with `IMM_BIT_EXTEND`, this single bit generates three essential constants without extra bus sources:
    - `0x00FF` (IMM=1, EXT_ZERO8): masks low byte of Rd (`Rd AND 0x00FF`) in T3 of LIH.
    - `0xFFFF` (IMM=1, EXT_SIGN8): tests register Rt via `Rt AND 0xFFFF` in T3 of BZ/BN to latch Z and N.
    - `0xFFFF = −1` (IMM=1, EXT_SIGN8): decrements SP via `SP + (−1)` in PUSH and increments SP via `SP − (−1)` in POP.

12. **Pre-shifted branch offsets in tara_decoder and ALU-A override in JMP_BZ_BN_CALL:** `tara_decoder` sign-extends rel8 and rel11 to 16 bits and shifts them left by 1 bit using two `shl16` instances. During JMP, CALL, and the taken state (T4) of BZ/BN, `JMP_BZ_BN_CALL` routes `REL_8` or `REL_11` onto ALU input A while PC drives the bus (ALU input B), computing `PC + 2 × rel` on the main ALU without needing a separate branch adder inside PC. Gating `JMP_BZ_BN_CALL` with `~STATUS_WE` ensures that during T3 of BZ/BN (when `STATUS_WE = 1`), ALU input A still reads Rt from QB.
13. **Hardware format steering for REG_A, REG_B, and REG_D:** Decoding FORMAT (F0–F7) in `taracpu` automatically routes RD as a source register for F3 (LIH, ADDI, SHL, SHR), F4 stores (STW, STB), F5 branches (BZ, BN), and F7 PUSH. It routes RA onto port A for F2 (MOV), so MOV completes in a single T3 step as `Rd ← R[RA]`, and it forces R6 (link register) for CALL/RET and R7 (SP) for PUSH/POP. POP additionally forces REG_A to R7, so its T3 step loads `MAR ← R7` directly from the stack pointer; the memory read, the two SP increments through ACC, and the final `Rd ← MDR` write-back then follow. Comparing `BUS_SOURCE == 5` (ACC) during F7 distinguishes SP write-backs (from ACC) from POP's final data register write-back (from MDR), saving a dedicated ROM control bit.
14. **Byte-lane multiplexing inside MDR rather than ram_word:** Keeping `ram_word` as a pure 16-bit word memory and placing byte-lane multiplexers inside MDR (controlled by `BYTE_OP` and `MAR_OUT[0]`) cleanly supports both zero-extended byte loads (LDB) on `MDR_BUS` and read-modify-write byte stores (STB) into MDR.
15. **16-bit control word reduction and hardware signal derivation:** In `G06_th6_tara.circ` and `G06_th6_microcode_gen.py`, signals that are strictly functions of the active T-state, opcode, or format (`IR_LOAD`, `PC_FETCH_EN`, `BYTE_OP`, and `ALU_OPCODE`) were moved from the ROM into combinational decode logic in `taracpu`. This reduced the control word from 25/24 bits to exactly 16 bits (4 hex digits), shrinking ROM storage, simplifying bus routing, and making the control unit substantially more elegant.
16. **Undefined opcodes execute as 4-cycle NOPs:** Unimplemented opcodes (MUL = 12 and 27–31) execute the fetch sequence (T0–T2) followed by SAFE (`STEP_RST = 1, BUS_OFF`) at T3. They modify no architectural state but do not halt the processor.

8. Test Programs & Expected Results

Six integration programs were written to exercise the complete TARA datapath, control ROM and memory system end to end. Each program is supplied as a Logisim `v2.0 raw` RAM image (`prog1_register_add.hex` … `prog6_array_max.hex`). Every image places code at byte address 0x0000 and data/results from byte address 0x0400. Programs use only `R0`–`R5` (R6 is the link register and R7 the stack pointer, so neither is disturbed) and finish with `HLT`. Expected values below were obtained by executing each image on an instruction-level model of the ISA derived from the microcode in Section 6 (instruction encodings follow the formats in Section 2), and each result was confirmed by running the image on the Logisim circuit; the screenshots are included as Figures 1–12.

8.1 Test Setup & Procedure

1. Open `G06_th6_tara.circ` with the control ROM `G06_cintrol_rom.hex` loaded (it is embedded in the circuit).
2. Right-click the RAM inside `ram_word` → *Load Image…* and select the program image (`progN_*.hex`). Logisim addresses the RAM by *word index*, which is the byte address ÷ 2 (e.g. byte address 0x0400 is word index 0x200); both are listed in every table below.
3. Reset the CPU (RST), then enable clock ticks (Simulate → Ticks Enabled) until the T-state counter freezes at T3 with `HALT` asserted.
4. Compare the register file and the listed RAM words (right-click RAM → *Edit Contents…*) with the *Expected outputs* table. The 32-bit `CYCLES` counter can be compared with the cycle count given for each program.

8.2 Summary of Test Programs

#	Program	Hex image	Inputs	Expected outputs	Instr.	Cycles
1	Register Addition	`prog1_register_add.hex`	none (registers loaded by code)	[0x0400] = 0x3E01, R3 = 0x3E01	9	48
2	Memory Subtraction	`prog2_memory_sub.hex`	[0x0400] = 0x2710, [0x0402] = 0x0D05	[0x0404] = 0x1A0B	7	40
3	Determination of Parity	`prog3_parity.hex`	[0x0400] = 0x1235, [0x0404] = 0x4A2C	[0x0402] = 0x0001, [0x0406] = 0x0000	15	79
4	Multiplication by Repeated Addition	`prog4_multiply.hex`	[0x0400] = 0x0125, [0x0402] = 0x0019	[0x0404] = 0x1C9D	108	545
5	Sum of a 16-bit Array	`prog5_array_sum.hex`	N = 8, A = {1000, 2500, 300, 4000, 12000, 50, 7000, 3} at 0x0402–0x0410	[0x0420] = 0x68E5	59	314
6	Maximum of a 16-bit Array	`prog6_array_max.hex`	N = 8, A = {1000, 2500, 300, 4000, 12000, 50, 7000, 3} at 0x0402–0x0410	[0x0420] = 0x2EE0	71	377

Note on cycle counts: `CYCLES` equals the sum of the per-instruction T-state counts in Section 2 (BZ/BN = 6 taken, 5 not taken), with `HLT` contributing 3 (T0–T2), because the counter stops when `HALT` is asserted in T3. The values listed are those read from the circuit (Figures 1, 3, 5, 7, 9, 11) and agree with the instruction-level model for all six programs. Programs 3 and 6 deliberately cover both branch outcomes of `BZ` and `BN`.

8.3 Program 1: Register Addition

Builds two 16-bit constants in registers using LIL + LIH, adds them with ADD (F1), and stores the sum to memory so the result can be checked both in the register file and in RAM. Exercises LIL, LIH (7-cycle read-modify-write of the upper byte), ADD, the base-register load idiom and STW.

Image to load: `prog1_register_add.hex` (9 words from word index 0; the zero-filled gap is run-length encoded). Code starts at byte address 0x0000; register `R5` is set to 0x0400 and used as the data base address.

Program listing

Addr	Word	Label	Instruction
0000	1934		LIL R1, 0x34
0002	2112		LIH R1, 0x12
0004	1ACD		LIL R2, 0xCD
0006	222B		LIH R2, 0x2B
0008	4B28		ADD R3, R1, R2
000A	1D00		LIL R5, 0x00
000C	2504		LIH R5, 0x04
000E	33A0		STW R3, [R5, 0]
0010	0800		HLT

Inputs (pre-loaded in RAM)

No data is pre-loaded in RAM; both operands are built in registers by the program itself (R1 = 0x1234, R2 = 0x2BCD).

Expected outputs after HLT

Location	RAM word idx	Expected (hex)	Expected (dec)	Meaning
[0x0400]	0x200	0x3E01	15873	result word
R3	—	0x3E01	15873	register result

Final register file (R0–R5; R6 and R7 are never touched): R0 = 0x0000, R1 = 0x1234, R2 = 0x2BCD, R3 = 0x3E01, R4 = 0x0000, R5 = 0x0400.
Run length: 9 instructions executed (including HLT); CYCLES = **48** at halt, as read from the circuit (Figure 1).

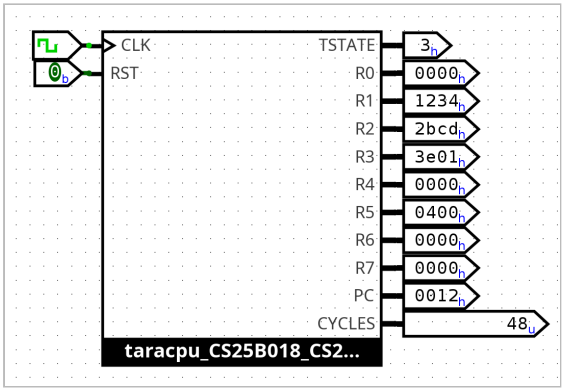


Figure 1: Program 1 (Register Addition) — registers R0–R7, PC and CYCLES after HLT (TSTATE = 3).

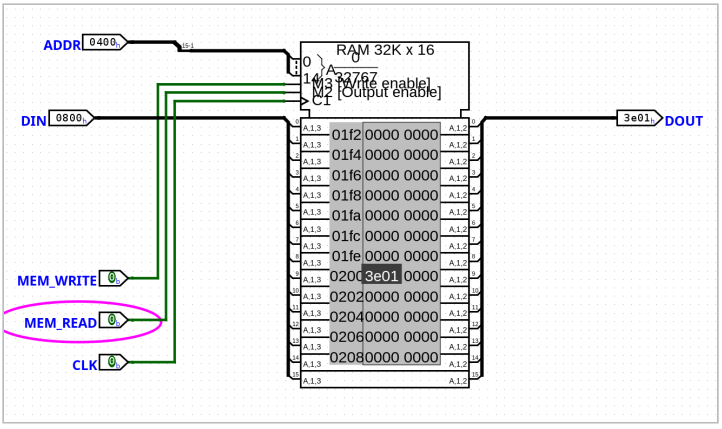


Figure 2: Program 1 — RAM after HLT: the stored sum 0x3E01 is in word 0x200 (byte address 0x0400). Each RAM row shows two consecutive words; row labels are word indices.

8.4 Program 2: Memory Subtraction

Loads two 16-bit operands from RAM with LDW, computes A − B with SUB (checks operand order Ra − Rb), and stores the difference with STW. Exercises LDW/STW word accesses at byte addresses 0x400–0x404.

Image to load: prog2_memory_sub.hex (514 words from word index 0; the zero-filled gap is run-length encoded). Code starts at byte address 0x0000; register R5 is set to 0x0400 and used as the data base address.

Program listing

Addr	Word	Label	Instruction
0000	1D00		LIL R5, 0x00

Addr	Word	Label	Instruction
0002	2504		LIH R5, 0x04
0004	29A0		LDW R1, [R5, 0]
0006	2AA2		LDW R2, [R5, 2]
0008	5328		SUB R3, R1, R2
000A	33A4		STW R3, [R5, 4]
000C	0800		HLT

Inputs (pre-loaded in RAM)

Byte addr	RAM word idx	Value (hex)	Value (dec)	Meaning
0x0400	0x200	0x2710	10000	A (minuend)
0x0402	0x201	0x0D05	3333	B (subtrahend)

Expected outputs after HLT

Location	RAM word idx	Expected (hex)	Expected (dec)	Meaning
[0x0404]	0x202	0x1A0B	6667	A − B

Final register file (R0–R5; R6 and R7 are never touched): `R0 = 0x0000`, `R1 = 0x2710`, `R2 = 0x0D05`, `R3 = 0x1A0B`, `R4 = 0x0000`, `R5 = 0x0400`.
Run length: 7 instructions executed (including HLT); `CYCLES` = **40** at halt, as read from the circuit (Figure 3).

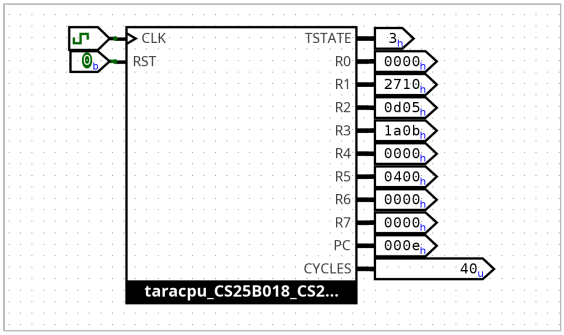


Figure 3: Program 2 (Memory Subtraction) — registers R0–R7, PC and CYCLES after HLT (TSTATE = 3).

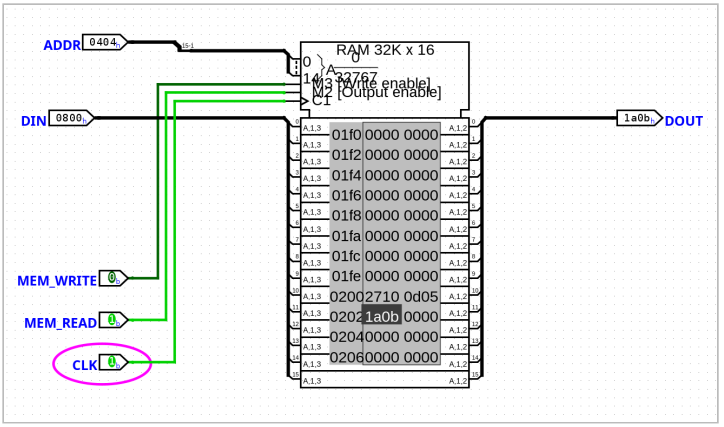


Figure 4: Program 2 — RAM after HLT: operands 0x2710 and 0x0D05 in words 0x200–0x201; difference 0x1A0B in word 0x202 (byte address 0x0404). Each RAM row shows two consecutive words; row labels are word indices.

8.5 Program 3: Determination of Parity

Determines whether each of two input words is odd or even by AND-ing with 1 and branching on the result with BZ. Writes 1 for odd and 0 for even. The two inputs make the branch exercised in *both* directions (BZ taken for the even input, not taken for the odd input).

Image to load: `prog3_parity.hex` (515 words from word index 0; the zero-filled gap is run-length encoded). Code starts at byte address 0x0000; register `R5` is set to 0x0400 and used as the data base address.

Program listing

Addr	Word	Label	Instruction
0000	1D00		LIL R5, 0x00
0002	2504		LIH R5, 0x04
0004	1A01		LIL R2, 0x01

Addr	Word	Label	Instruction
0006	29A0		LDW R1, [R5, 0]
0008	6B28		AND R3, R1, R2
000A	1C00		LIL R4, 0x00
000C	A301		BZ R3, L1
000E	1C01		LIL R4, 0x01
0010	34A2	L1	STW R4, [R5, 2]
0012	29A4		LDW R1, [R5, 4]
0014	6B28		AND R3, R1, R2
0016	1C00		LIL R4, 0x00
0018	A301		BZ R3, L2
001A	1C01		LIL R4, 0x01
001C	34A6	L2	STW R4, [R5, 6]
001E	0800		HLT

Inputs (pre-loaded in RAM)

Byte addr	RAM word idx	Value (hex)	Value (dec)	Meaning
0x0400	0x200	0x1235	4661	input 1 (odd)
0x0404	0x202	0x4A2C	18988	input 2 (even)

Expected outputs after HLT

Location	RAM word idx	Expected (hex)	Expected (dec)	Meaning
[0x0402]	0x201	0x0001	1	parity of input 1 (1 = odd)
[0x0406]	0x203	0x0000	0	parity of input 2 (0 = even)

Final register file (R0–R5; R6 and R7 are never touched): `R0 = 0x0000`, `R1 = 0x4A2C`, `R2 = 0x0001`, `R3 = 0x0000`, `R4 = 0x0000`, `R5 = 0x0400`.
Run length: 15 instructions executed (including HLT); `CYCLES` = **79** at halt, as read from the circuit (Figure 5).

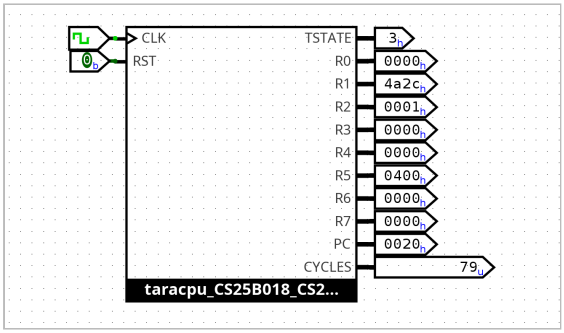


Figure 5: Program 3 (Determination of Parity) — registers R0–R7, PC and CYCLES after HLT (TSTATE = 3).

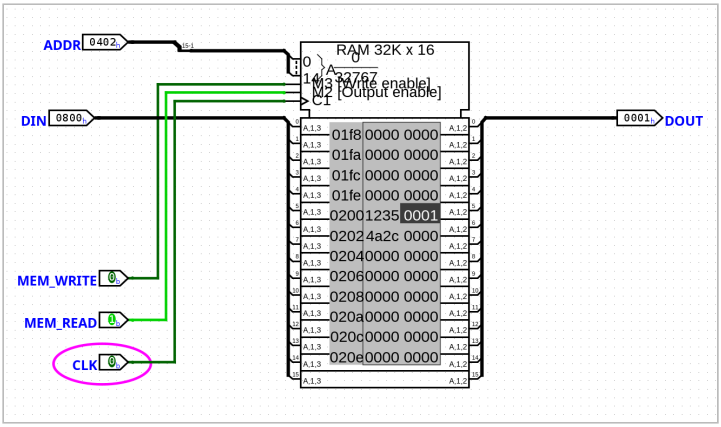


Figure 6: Program 3 — RAM after HLT: inputs 0x1235 and 0x4A2C in words 0x200 and 0x202; parity results 0x0001 (odd) and 0x0000 (even) in words 0x201 and 0x203. Each RAM row shows two consecutive words; row labels are word indices.

8.6 Program 4: Multiplication by Repeated Addition

Computes A × B without the (unimplemented) MUL opcode by adding the multiplicand A to an accumulator B times, using ADDI −1 as the loop counter, BZ as the exit test and JMP (negative offset) as the loop-back. Exercises a

long loop of mixed-length instructions and backward JMP.

Image to load: `prog4_multiply.hex` (514 words from word index 0; the zero-filled gap is run-length encoded). Code starts at byte address 0x0000; register `R5` is set to 0x0400 and used as the data base address.

Program listing

Addr	Word	Label	Instruction
0000	1D00		LIL R5, 0x00
0002	2504		LIH R5, 0x04
0004	29A0		LDW R1, [R5, 0]
0006	2AA2		LDW R2, [R5, 2]
0008	1B00		LIL R3, 0x00
000A	A203	loop	BZ R2, done
000C	4B64		ADD R3, R3, R1
000E	5AFF		ADDI R2, -1
0010	B7FC		JMP loop
0012	33A4	done	STW R3, [R5, 4]
0014	0800		HLT

Inputs (pre-loaded in RAM)

Byte addr	RAM word idx	Value (hex)	Value (dec)	Meaning
0x0400	0x200	0x0125	293	A (multiplicand)
0x0402	0x201	0x0019	25	B (multiplier / loop count)

Expected outputs after HLT

Location	RAM word idx	Expected (hex)	Expected (dec)	Meaning
[0x0404]	0x202	0x1C9D	7325	A × B

Final register file (R0–R5; R6 and R7 are never touched): `R0 = 0x0000`, `R1 = 0x0125`, `R2 = 0x0000`, `R3 = 0x1C9D`, `R4 = 0x0000`, `R5 = 0x0400`.
Run length: 108 instructions executed (including HLT); `CYCLES` = **545** at halt, as read from the circuit (Figure 7).

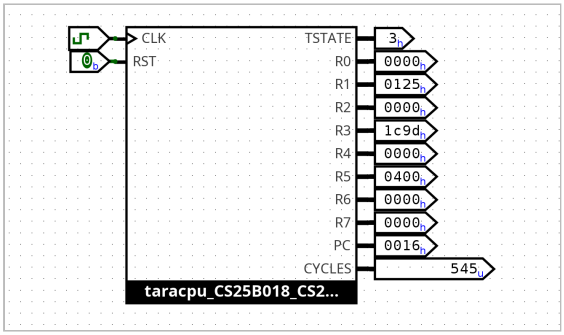


Figure 7: Program 4 (Multiplication by Repeated Addition) — registers R0–R7, PC and CYCLES after HLT (TSTATE = 3).

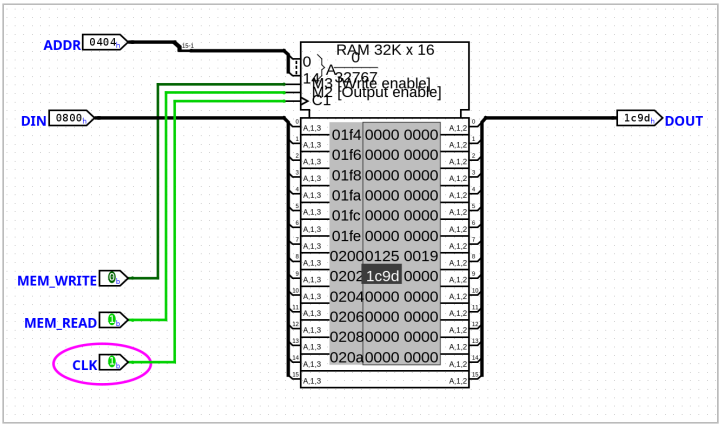


Figure 8: Program 4 — RAM after HLT: multiplicand 0x0125 and multiplier 0x0019 in words 0x200–0x201; product 0x1C9D in word 0x202 (byte address 0x0404). Each RAM row shows two consecutive words; row labels are word indices.

8.7 Program 5: Sum of a 16-bit Array

Sums N 16-bit elements starting at 0x0402 using a moving pointer (R4, stepped by 2 bytes) and a down-counter (R1). The sum is written to a fixed result word at 0x0420. Exercises LDW with offset 0, register-pointer arithmetic with ADDI, a counted loop and a 16-bit accumulator.

Image to load: `prog5_array_sum.hex` (521 words from word index 0; the zero-filled gap is run-length encoded). Code starts at byte address 0x0000; register `R5` is set to 0x0400 and used as the data base address.

Program listing

Addr	Word	Label	Instruction
0000	1D00		LIL R5, 0x00
0002	2504		LIH R5, 0x04
0004	29A0		LDW R1, [R5, 0]
0006	1A00		LIL R2, 0x00
0008	14A0		MOV R4, R5
000A	5C02		ADDI R4, 2
000C	A105	loop	BZ R1, done
000E	2B80		LDW R3, [R4, 0]
0010	4A4C		ADD R2, R2, R3
0012	5C02		ADDI R4, 2
0014	59FF		ADDI R1, -1
0016	B7FA		JMP loop
0018	1820	done	LIL R0, 0x20
001A	2004		LIH R0, 0x04
001C	3200		STW R2, [R0, 0]
001E	0800		HLT

Inputs (pre-loaded in RAM)

Byte addr	RAM word idx	Value (hex)	Value (dec)	Meaning
0x0400	0x200	0x0008	8	N (element count)
0x0402	0x201	0x03E8	1000	A[0]
0x0404	0x202	0x09C4	2500	A[1]
0x0406	0x203	0x012C	300	A[2]
0x0408	0x204	0x0FA0	4000	A[3]
0x040A	0x205	0x2EE0	12000	A[4]
0x040C	0x206	0x0032	50	A[5]
0x040E	0x207	0x1B58	7000	A[6]
0x0410	0x208	0x0003	3	A[7]

Expected outputs after HLT

Location	RAM word idx	Expected (hex)	Expected (dec)	Meaning
[0x0420]	0x210	0x68E5	26853	sum of A[0..7]

Final register file (R0–R5; R6 and R7 are never touched): `R0 = 0x0420`, `R1 = 0x0000`, `R2 = 0x68E5`, `R3 = 0x0003`, `R4 = 0x0412`, `R5 = 0x0400`.
Run length: 59 instructions executed (including HLT); `CYCLES` = **314** at halt, as read from the circuit (Figure 9).

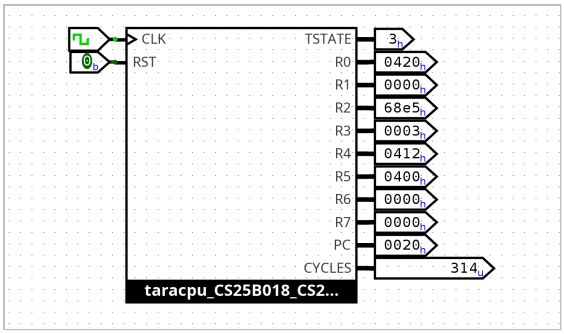


Figure 9: Program 5 (Sum of a 16-bit Array) — registers R0–R7, PC and CYCLES after HLT (TSTATE = 3).

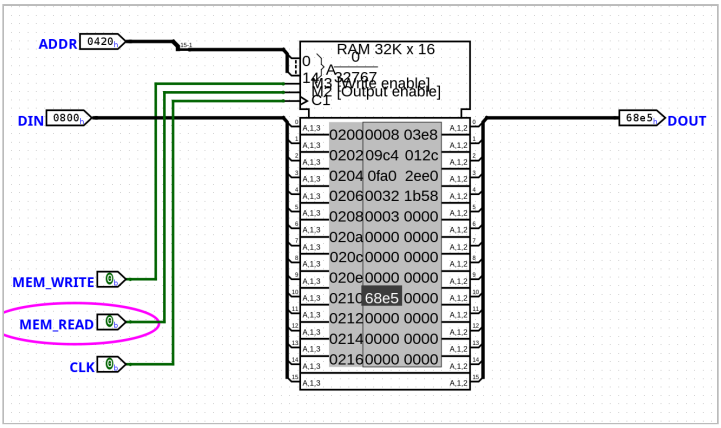


Figure 10: Program 5 — RAM after HLT: N = 8 and the array in words 0x200–0x208; sum 0x68E5 in word 0x210 (byte address 0x0420). Each RAM row shows two consecutive words; row labels are word indices.

8.8 Program 6: Maximum of a 16-bit Array

Finds the largest of N 16-bit elements (same array as Program 5). The running maximum is initialised with A[0]; each element is compared by computing cur − max with SUB and testing the sign with BN (taken ⇒ keep the old maximum, otherwise MOV the new one in). The array values are all < 0x8000 so the subtraction cannot overflow. The maximum is stored at 0x0420.

Image to load: `prog6_array_max.hex` (521 words from word index 0; the zero-filled gap is run-length encoded). Code starts at byte address 0x0000; register `R5` is set to 0x0400 and used as the data base address.

Program listing

Addr	Word	Label	Instruction
0000	1D00		LIL R5, 0x00
0002	2504		LIH R5, 0x04
0004	29A0		LDW R1, [R5, 0]
0006	14A0		MOV R4, R5
0008	5C02		ADDI R4, 2
000A	2A80		LDW R2, [R4, 0]
000C	A107	loop	BZ R1, done
000E	2B80		LDW R3, [R4, 0]
0010	5068		SUB R0, R3, R2
0012	A801		BN R0, skip
0014	1260		MOV R2, R3
0016	5C02	skip	ADDI R4, 2
0018	59FF		ADDI R1, -1
001A	B7F8		JMP loop
001C	1820	done	LIL R0, 0x20
001E	2004		LIH R0, 0x04
0020	3200		STW R2, [R0, 0]
0022	0800		HLT

Inputs (pre-loaded in RAM)

Byte addr	RAM word idx	Value (hex)	Value (dec)	Meaning
0x0400	0x200	0x0008	8	N (element count)

Byte addr	RAM word idx	Value (hex)	Value (dec)	Meaning
0x0402	0x201	0x03E8	1000	A[0]
0x0404	0x202	0x09C4	2500	A[1]
0x0406	0x203	0x012C	300	A[2]
0x0408	0x204	0x0FA0	4000	A[3]
0x040A	0x205	0x2EE0	12000	A[4]
0x040C	0x206	0x0032	50	A[5]
0x040E	0x207	0x1B58	7000	A[6]
0x0410	0x208	0x0003	3	A[7]

Expected outputs after HLT

Location	RAM word idx	Expected (hex)	Expected (dec)	Meaning
[0x0420]	0x210	0x2EE0	12000	max of A[0..7]

Final register file (R0–R5; R6 and R7 are never touched): `R0 = 0x0420`, `R1 = 0x0000`, `R2 = 0x2EE0`, `R3 = 0x0003`, `R4 = 0x0412`, `R5 = 0x0400`.
Run length: 71 instructions executed (including HLT); `CYCLES` = **377** at halt, as read from the circuit (Figure 11).

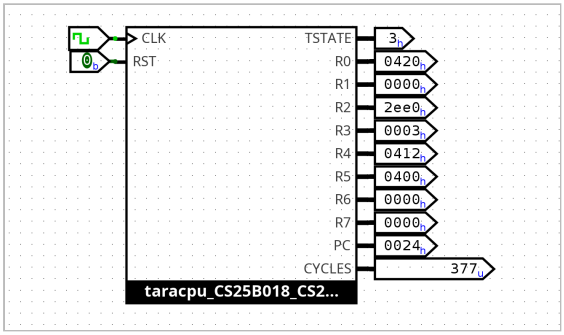


Figure 11: Program 6 (Maximum of a 16-bit Array) — registers R0–R7, PC and CYCLES after HLT (TSTATE = 3).

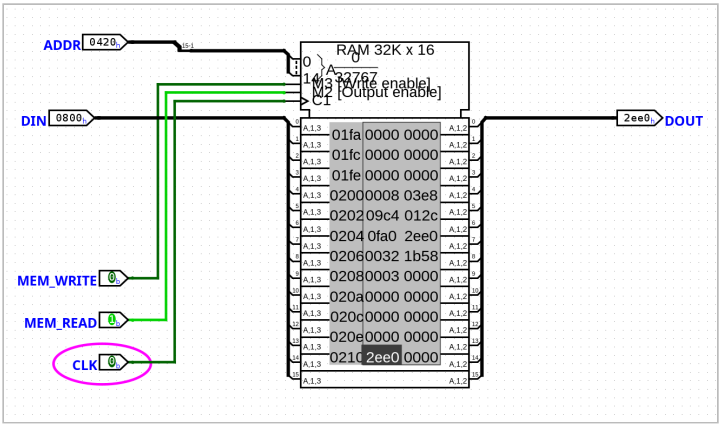


Figure 12: Program 6 — RAM after HLT: N = 8 and the array in words 0x200–0x208; maximum 0x2EE0 in word 0x210 (byte address 0x0420). Each RAM row shows two consecutive words; row labels are word indices.

9. Faults Encountered & Architectural Limitations

9.1 Faults Encountered and Resolved

1. **Incorrect COUT inversion on SUB in the ALU:** An extra NOT gate was initially placed on COUT for subtraction; removing it restored the true carry-out from `addsub16`.
2. **Word-indexed vs. byte-addressed ram_word:** ADDR was initially wired directly to a 16-bit-addressed RAM primitive, causing 0x0000 and 0x0001 to access distinct words. Dropping `ADDR[0]` via a splitter and reducing the RAM to `addrWidth = 15` fixed byte addressing.
3. **Missing reset on ACC and redundant EXT mux in regfile8x16:** The first datapath version lacked a reset connection on ACC; wiring `PC_RST` to ACC and removing the temporary EXT loopback mux unified all register writes through the shared bus.
4. **Reversed operand order on non-commutative ALU instructions:** Wiring RA to `REG_A` (bus → ALU B) and RB to `REG_B` (ALU A) caused SUB to compute `RB − RA` (0xFFF8 instead of 0x0008). Swapping RA to `REG_B` and RB to `REG_A` fixed SUB, SLT, SHL, SHR, and NOT.
5. **Stale memory read during T1 fetch:** Because `ram_word` latched on the rising clock edge, MDR captured the previous memory output at the end of T1, causing the first instruction to execute as a NOP. Inverting the

ram_word clock in tara_datapath resolved the timing hazard within a 3-cycle fetch.

9.2 Known Architectural Limitations

1. **Unaligned word accesses are silently word-aligned:** Because ram_word ignores ADDR[0], executing LDW, STW, PUSH, or POP with an odd address accesses the 16-bit word at ADDR & 0xFFFE without raising an alignment exception.
2. **Two-step stack pointer updates in PUSH and POP:** Because the constant generator provides −1 (0xFFFF) via IMM = 1, EXT_SIGN8, PUSH decrements R7 twice (T4–T7, 9 cycles total) and POP increments R7 twice (T6–T9, 11 cycles total). However, because PUSH latches MDR ← Rs in T3 before modifying R7 and POP writes Rd ← MDR in T10 after updating R7, PUSH R7 pushes the original SP and POP R7 retains the popped word.
3. **Manual Link Register Management for Recursive Calls:** The CALL instruction unconditionally overwrites R6 with the return address. Nested or recursive subroutine calls will corrupt the link register unless the programmer explicitly saves R6 onto the stack (via PUSH R6) prior to recursive invocation and restores it (via POP R6) before executing RET.
4. **STB stores only the low byte of the source register:** STB always writes Rs[7:0] into the byte lane selected by MAR[0], via the MDR byte-lane merge at T6. There is no form that stores Rs[15:8] directly. To store a register's upper byte, software must first copy it into a scratch register and shift it down, for example MOV Rt, Rs, then SHR Rt, 8, then STB Rt, [Rb + off5]. STB is also a read-modify-write: it reads the full 16-bit word at T5, merges one byte at T6 and writes the word back at T7, which takes 8 cycles. The other byte of the word is preserved, and nothing else can access memory between the read and the write on this single-master design.

10. RAM_WORD and MDR Byte-Lane Implementation

Memory is organized as 32K × 16-bit little-endian words inside ram_word, while byte selection for LDB and byte merging for STB are implemented inside MDR using BYTE_OP (hardware-derived in taracpu at T6 for opcodes 7 and 8) and ADDR0 = MAR_OUT[0]:

Operation	BYTE_OP	ADDR0	MDR Input Path (MDR_IN ← register16)	MDR Bus Output Path (register16 ← MDR_BUS)
Word Read / Write (FETCH, LDW, STW, PUSH, POP, and T5 of LDB/STB)	0	0 or 1	MDR ← MDR_IN[15:0] (captures full 16-bit word from bus or ram_word)	MDR_BUS ← MDR_OUT[15:0] (drives full 16-bit word onto bus)
Even Byte Load / Merge (LDB at T6, STB merge at T6)	1	0 (even)	MDR[15:8] ← MDR_OUT[15:8] (preserved) MDR[7:0] ← MDR_IN[7:0] (Rs[7:0])	MDR_BUS ← {0x00, MDR_OUT[7:0]} (zero-extended low byte)
Odd Byte Load / Merge (LDB at T6, STB merge at T6)	1	1 (odd)	MDR[15:8] ← MDR_IN[7:0] (Rs[7:0]) MDR[7:0] ← MDR_OUT[7:0] (preserved)	MDR_BUS ← {0x00, MDR_OUT[15:8]} (zero-extended high byte)

Consequently, LDB reads the full word into MDR at T5 (BYTE_OP = 0) and drives the zero-extended byte onto the bus into Rd at T6 (BYTE_OP = 1). STB performs a clean read-modify-write: it reads the existing 16-bit word into MDR at T5 (BYTE_OP = 0), replaces only the addressed byte lane with Rs[7:0] at T6 (BYTE_OP = 1, BUS_SOURCE = RF, MDR_LOAD = 1), and writes the merged 16-bit word from MDR_OUT back to ram_word at T7 (MEM_WRITE = 1).

11. Verification and Consistency Checks

The control ROM generator G06_th6_microcode_gen.py builds the ROM from symbolic field definitions and guarantees architectural correctness by enforcing the following formal design invariants:

- **Bus Exclusivity:** Exactly one bus driver is enabled in any state (`BUS_SOURCE ∈ {0..7}`), preventing tri-state bus collisions.
- **Termination Guarantee:** Every implemented instruction path terminates in either `STEP_RST = 1` (returning to T0) or `HALT = 1` (freezing the counter). No instruction can loop indefinitely in microcode.
- **Memory Bus Safety:** `MEM_READ` and `MEM_WRITE` are strictly mutually exclusive and asserted only during memory transfer cycles.
- **Valid Write Sources:** Register and memory write enables (`REG_WE`, `MEM_WRITE`) are asserted only when a valid driver is driving the bus.
- **Quadrant Consistency for Non-Branch Opcodes:** For all opcodes other than BZ and BN, all four condition code quadrants (Z, N) contain identical microcode words.
- **Safe Idle Execution:** Unimplemented opcodes execute the fetch sequence followed by `SAFE = 0x3801`, ensuring they act as harmless 4-cycle NOPs without modifying state.
- **Uniform Fetch:** Fetch states T0, T1, and T2 are identical across all 32 opcodes and all flag quadrants.
- **Legal PC Updates:** `PC_LOAD` is asserted exclusively during fetch T1 for sequential increment, and during target load states for branches, jumps, CALL, and RET.
- **16-Bit Word Bound:** Every control word strictly fits within the 16-bit ROM data width (`word ≤ 0xFFFF`).
- **Hardware Signal Consistency:** Hardware generators in `taracpu` for `IR_LOAD`, `PC_FETCH_EN`, `BYTE_OP`, and `ALU_OPCODE` match the microcode requirements exactly across all 2048 ROM locations.